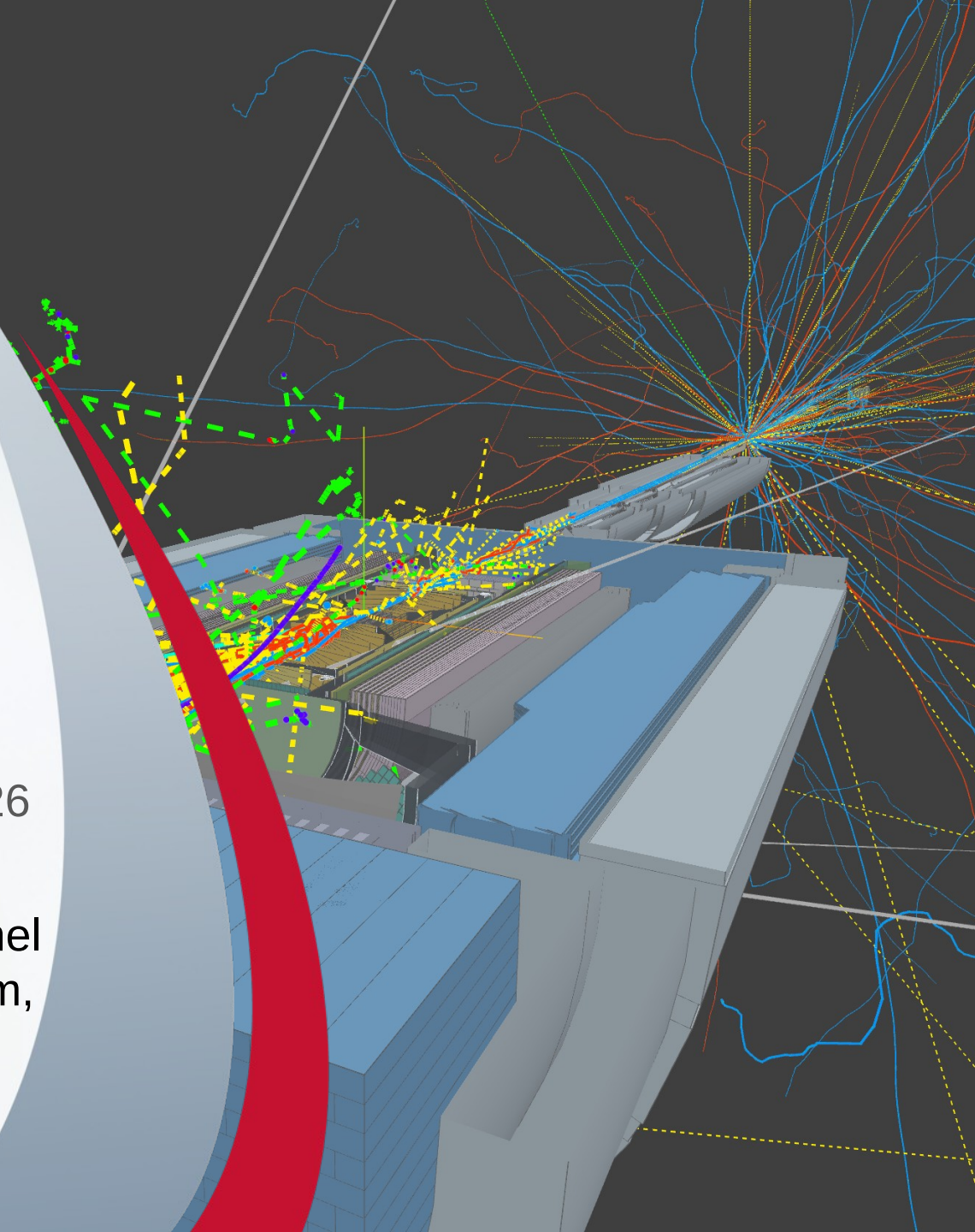
Streaming Readout AI Background Suppression



EIC ePIC collaboration meeting

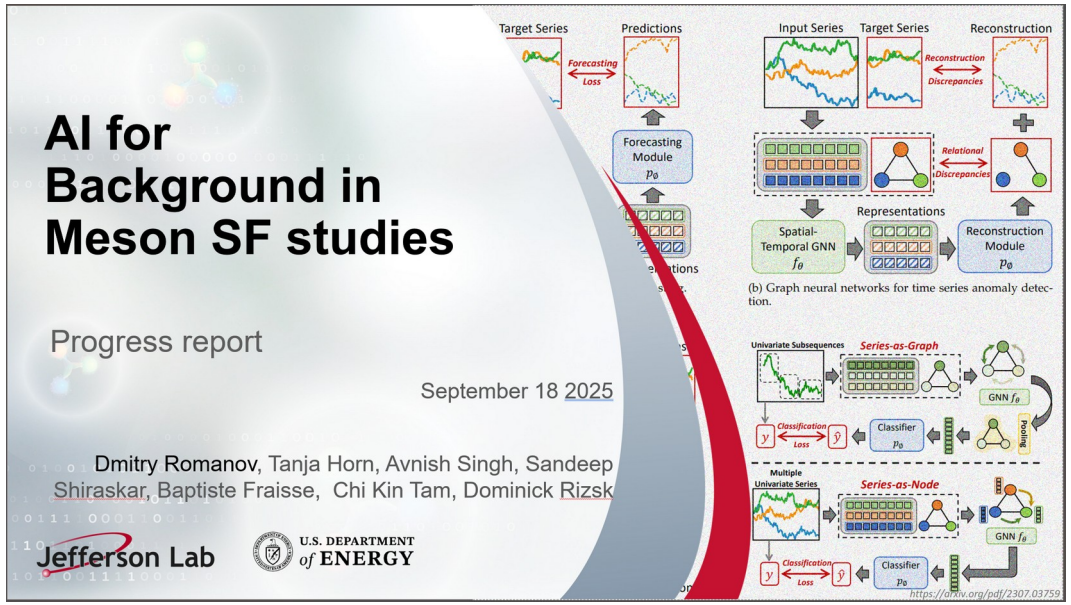
July 15 2026

Talk: **Dmitry Romanov**,
Team: Tanja Horn, Baptiste Fraisse, Avnish Singh, Rachel Montgomery, Antonio Perillo, Gary Penman, Chi Kin Tam, and Meson Structure Functions team

Jefferson Lab

U.S. DEPARTMENT of ENERGY

Our experience with Background Data



Primary Goal: Demonstrate that an AI-driven analysis of streaming-like frame-based data can:

1. Improve the identification of key physics events in high-background environments
2. Increase the throughput of scientific results

Key steps:

- Train AI to clean the background data
- Quantify the performance over traditional methods:
  - Discriminate events in physics background
  - Discriminate detector background
- Integrate models within JANA2 / Epic reconstruction

Data processing progression and AI/ML

Readout
Synchronization
Formation
Pre-filtration

Tracking
PID
Calorimeters
Reconstruction

Electron finder
DIS calculation
Analysis



This talk

Quick review

Physics + background data

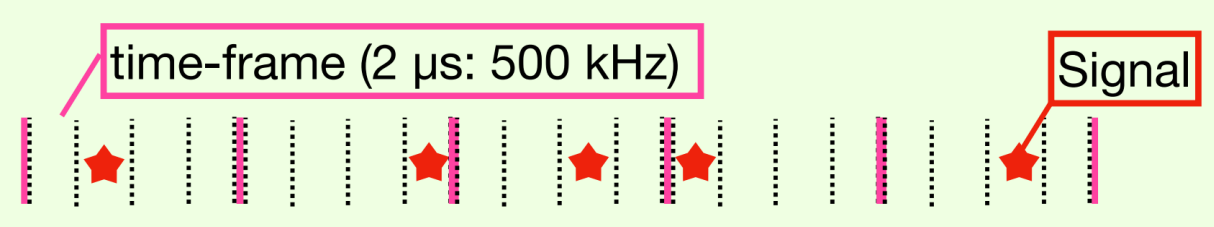
Event	signal	synrad	ebrems	etouschek	ecoloumb	p.b.gas
Event 1	●	● ● ● ●	●			
Event 2	●	● ● ● ●			●	●
Event 3	●	● ● ●		●		
Event 4	●	● ● ●	●			●
Event 5	●	● ● ● ●	●			

[Background mixed samples]

[Background wiki]

Legend

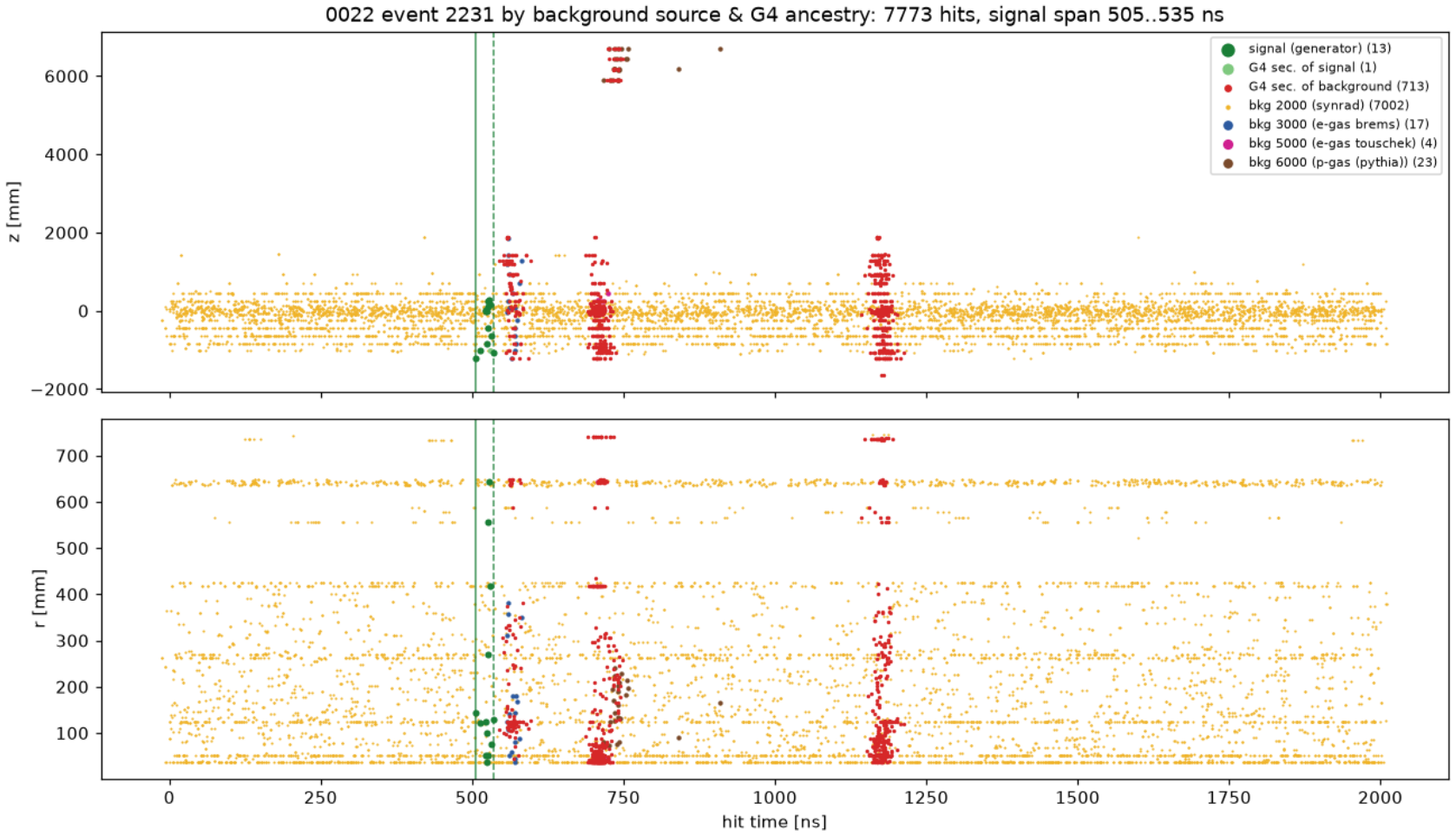
Symbol	Process	Description	Sampling Frequency (kHz)	Status Code Shift
●	**signal**	DIS NC 18x275 Q^2>1 (Deep inelastic scattering neutral current)	500	0
●	**synrad**	Synchrotron Radiation	14000	2000
●	**ebrems**	Electron bremsstrahlung radiation	316.94	3000
●	**etouschek**	Electron Touschek scattering (intrabeam scattering)	1.3	4000
●	**ecoulomb**	Electron Coulomb scattering processes	0.72	5000
●	**p.b.gas**	Proton beam gas interactions	22.5	6000



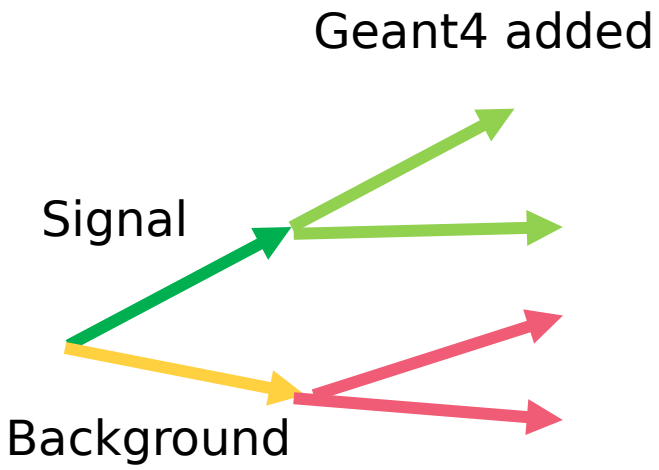
The most event looks like this

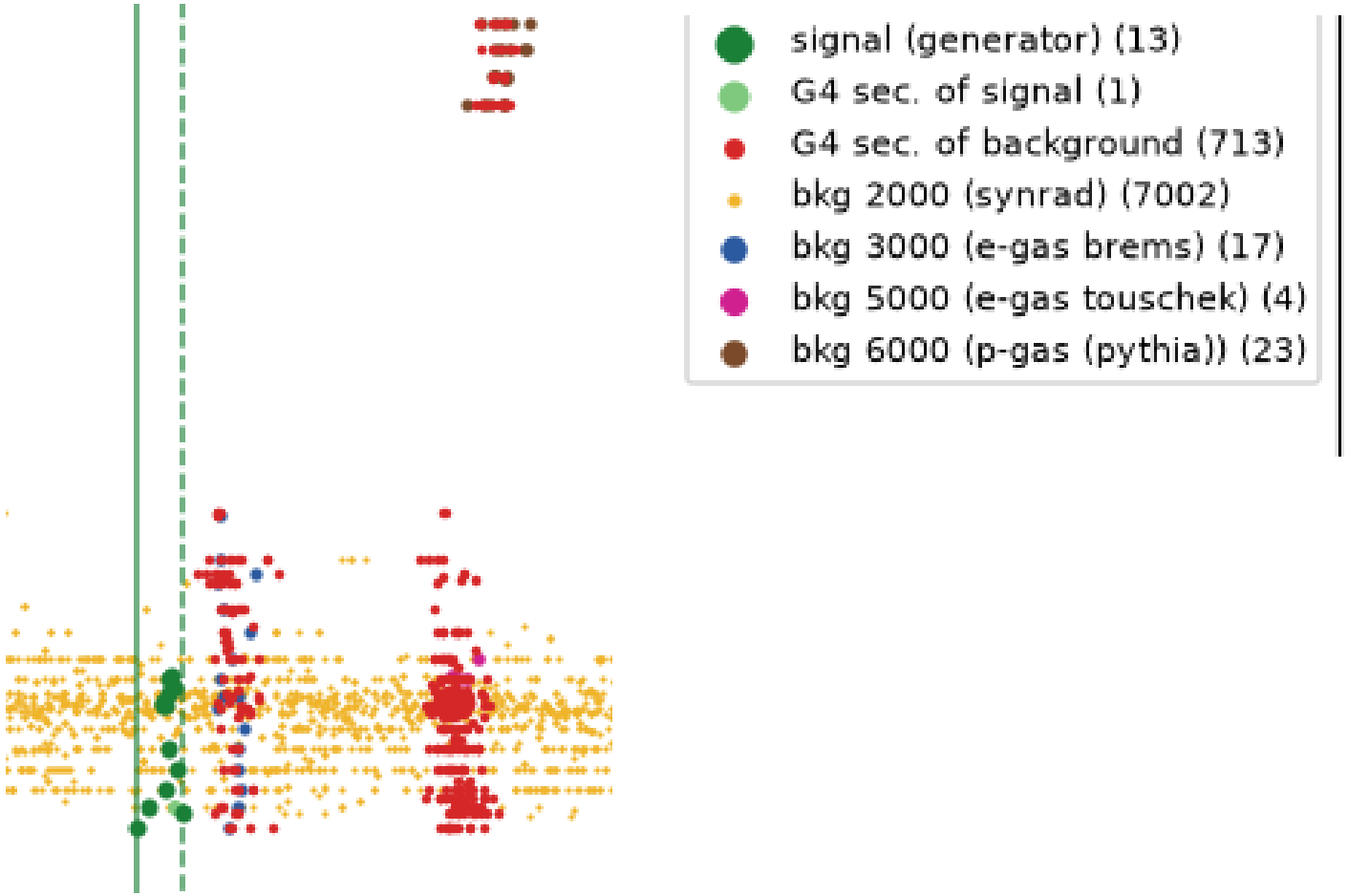


Very rare event with different backgrounds



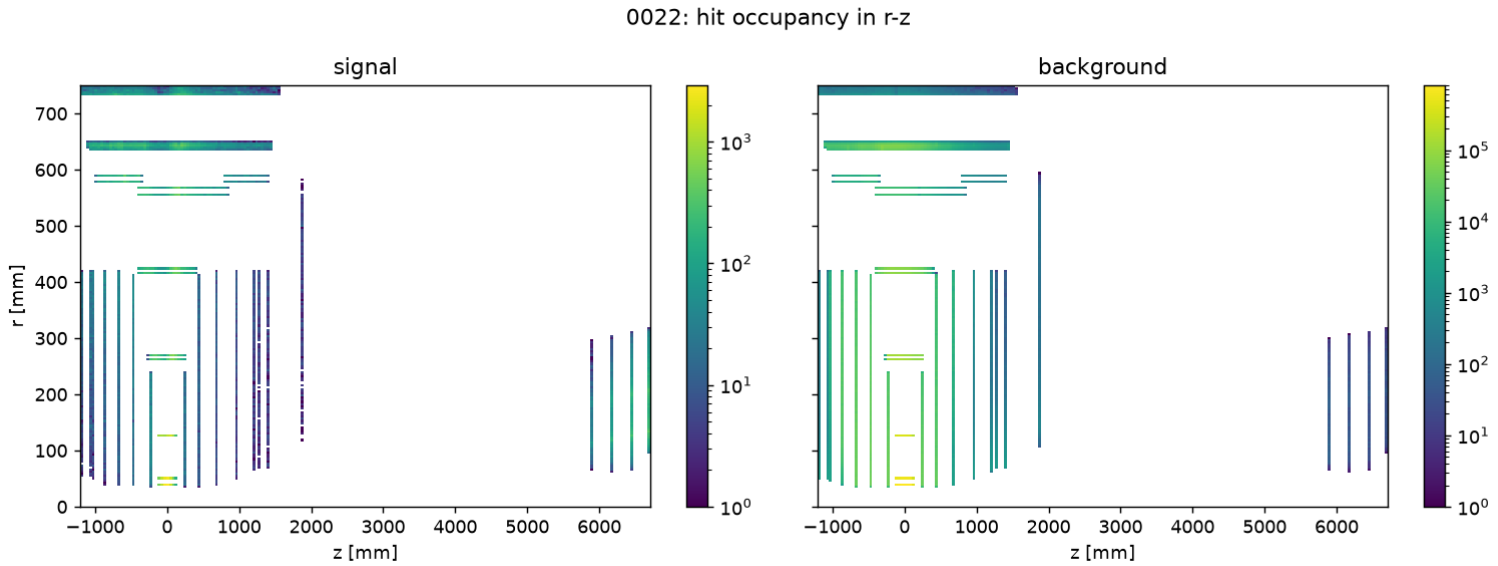
What are those dots



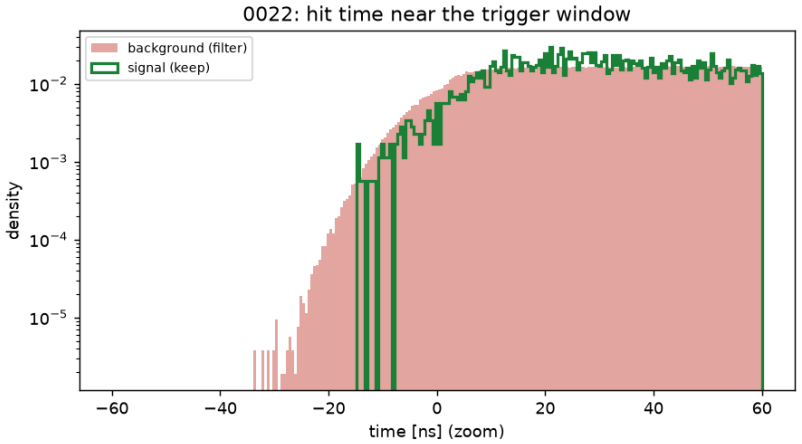


- Gean4 adds particles
- status code 0 be it signal or background,
- have to check where things come from

The data: a 2000 ns haystack, a 0.5% needle



0022_occupancy_rz.png

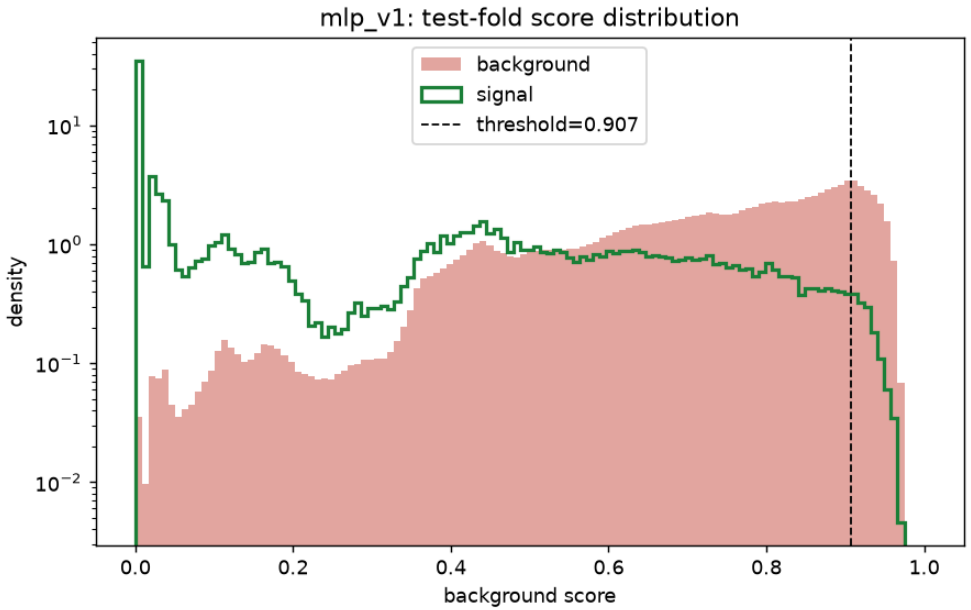


0022_time_zoom.png

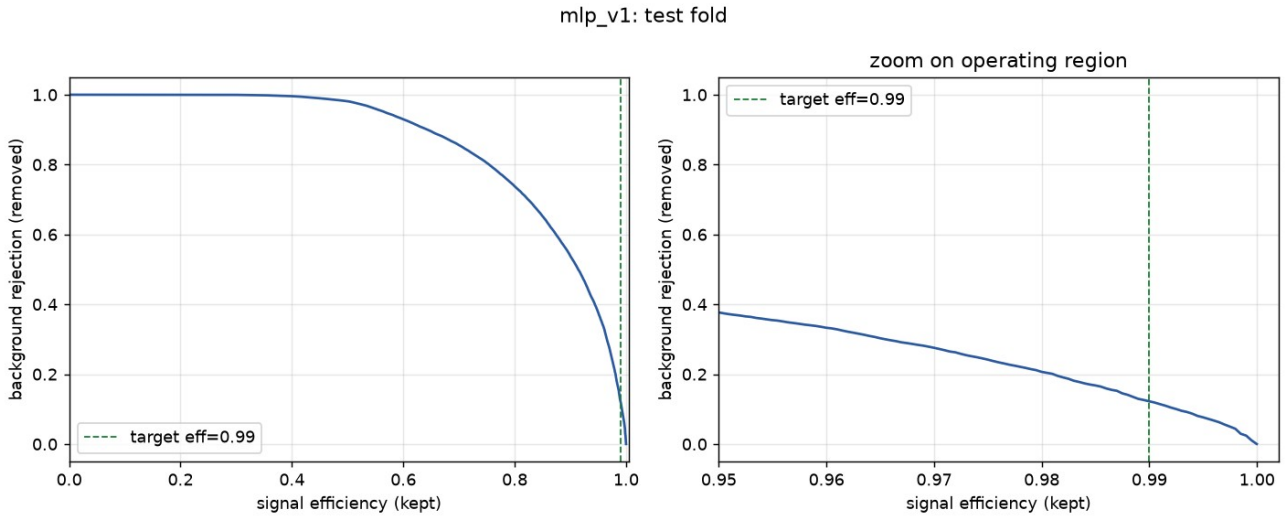
- ~35M hits per file; signal is only ~0.5% of hits
- Background dominated by synchrotron-radiation haze
(97% of hits) plus beam-gas showers
- **Per-hit inputs only: x, y, z, t, detector system id**
- One interaction per frame, at a uniformly random t0

A naive per-hit DNN looks good…

- 13k-parameter MLP on (x, y, z, t, r + system embedding), scored per hit
- AUC 0.864; high rejection efficiency! (false)
- Score separation looks convincing



mlp_v1_scores.png

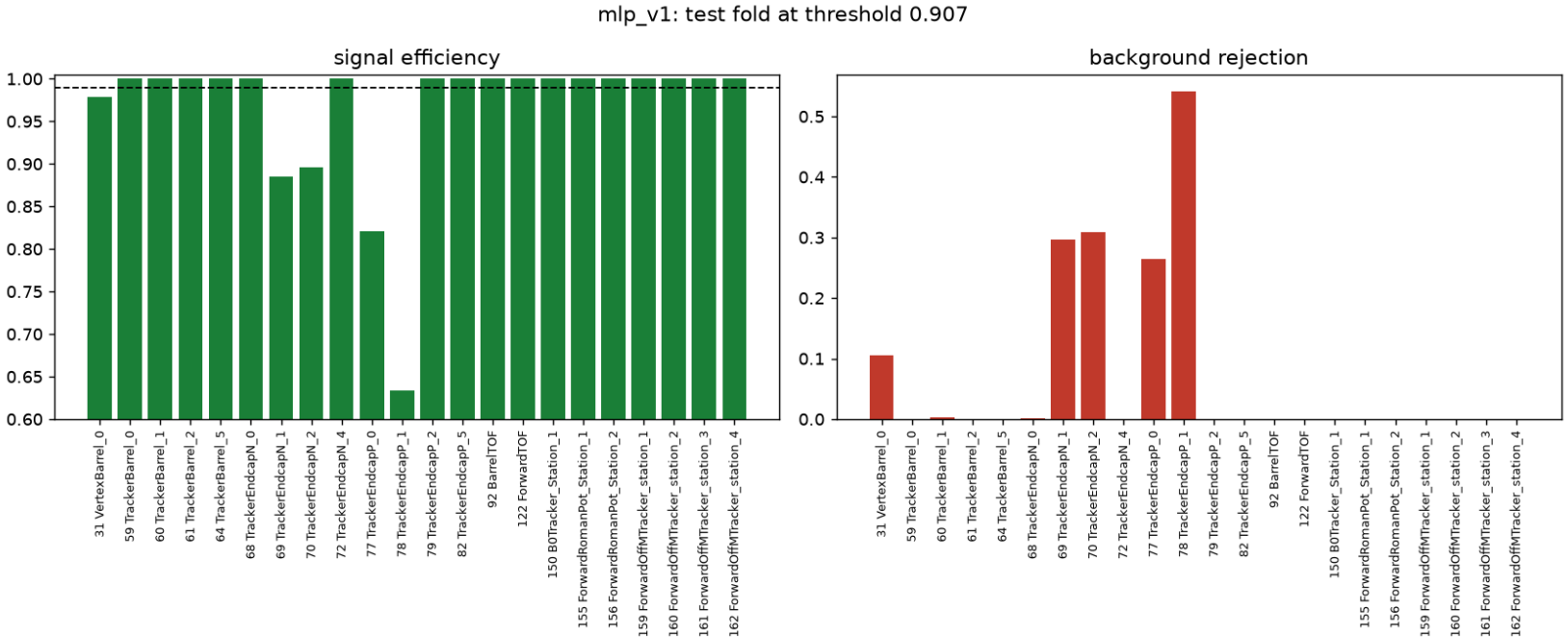


mlp_v1_eff_rejection.png

(This is example of of bad results but that looked promising in the beginning)

…but per subsystem it is quietly destructive

- A single global threshold silently kills 12–37% of signal in the inner tracker-endcap disks
  – systems 69/70 (TrackerEndcapN) and 77/78 (TrackerEndcapP); the central barrel is fine
- **Global metrics hide local damage**

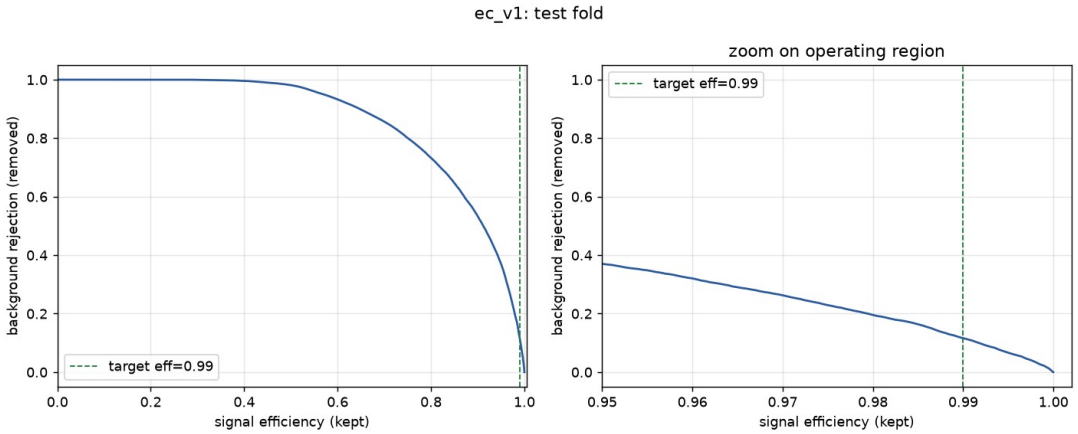


(This is example of actually bad results)

The per-hit ceiling: models doesn't help

- **Prioritize keeping signal over background rejection (around 99% or better)**
- 20× bigger MLP: 14.4% rejection.
- DeepSets with global event context: 13.6%.
- No per-hit feature set gets further
- Why: an individual signal hit and an individual background hit look IDENTICAL

Model	Signal efficiency	Background rejection
mlp_v1 (13k params)	98.8%	14.1%
mlp_v2 (273k params)	98.8%	14.4%
DeepSets (event context)	98.8%	13.6%



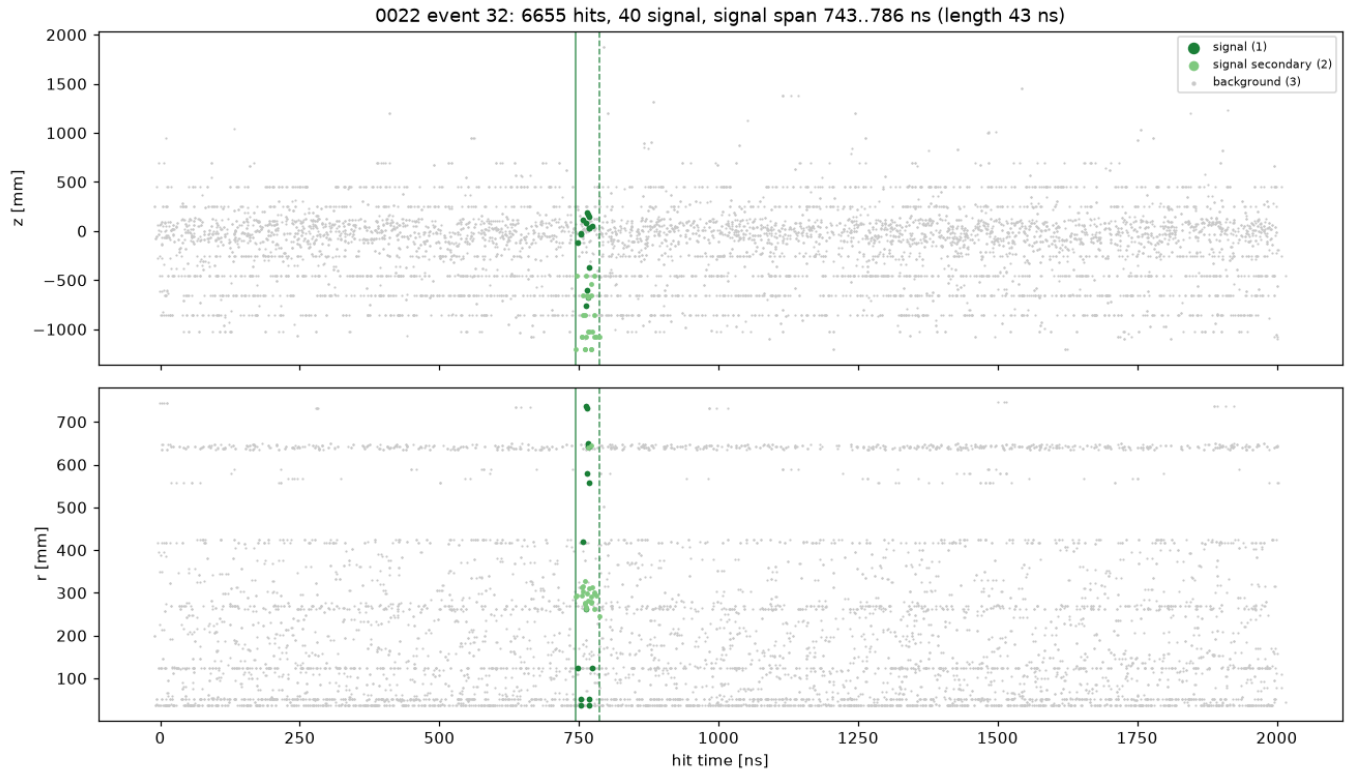
ec_v1_eff_rejection.png

(This is example of actually bad results)

Hunt for ideal window finder begins!

- The target: ≥99% of signal hits must survive!

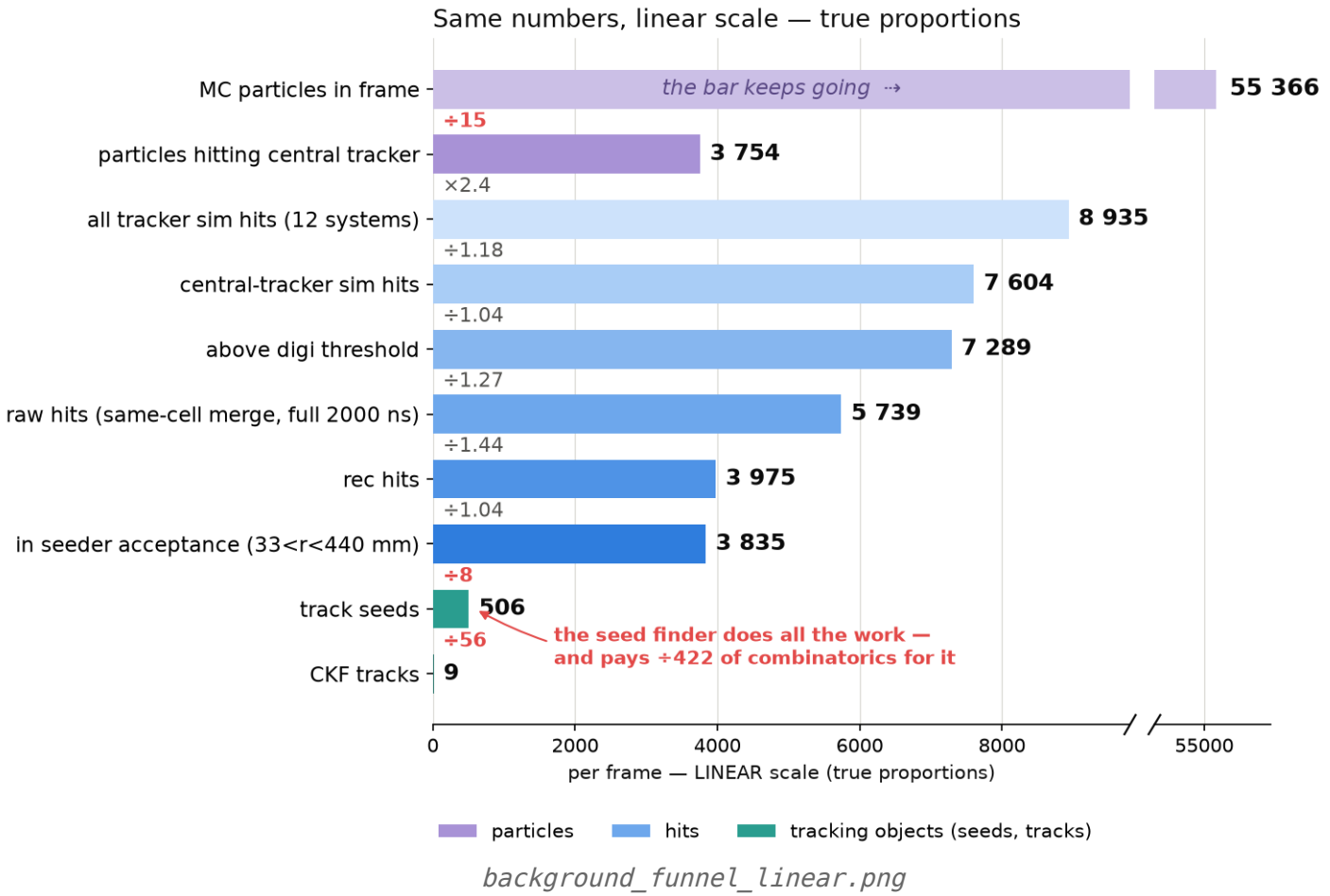
pipeline	signal efficiency	background rejection
Best per-hit classifier - no time window	**98.8%**	**14.1%**
PERFECT window, same hit models (the prize)	***99.0%***	***97.5%***



Apparently,
finding good window with signal:

1. Task, that we had to do in all streaming projects and prototypes
2. Allows to use complex networks
3. Gets near perfect results

But how EICrecon tracking lives without windows?



background_funnel_linear.png

- **Stock chain never uses hit time:**

  – digitization integrates the full 2000 ns frame (same-cell hits merged)

  – ACTS seeder reads only (x, y, z); seed time is hard-coded to 10 ns

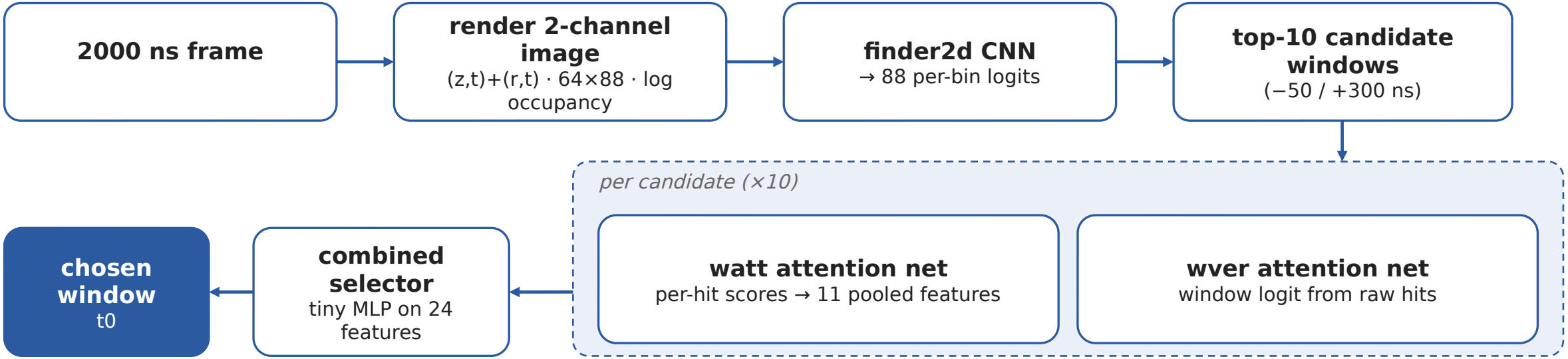
  – CKF fits 2 local coordinates; time is stored but unused

- **The background killer is GEOMETRY: a seed triplet must fit a helix pointing to the beamline (|d0| < 3 mm, |z| < 250 mm, pT ≳ 100 MeV)**

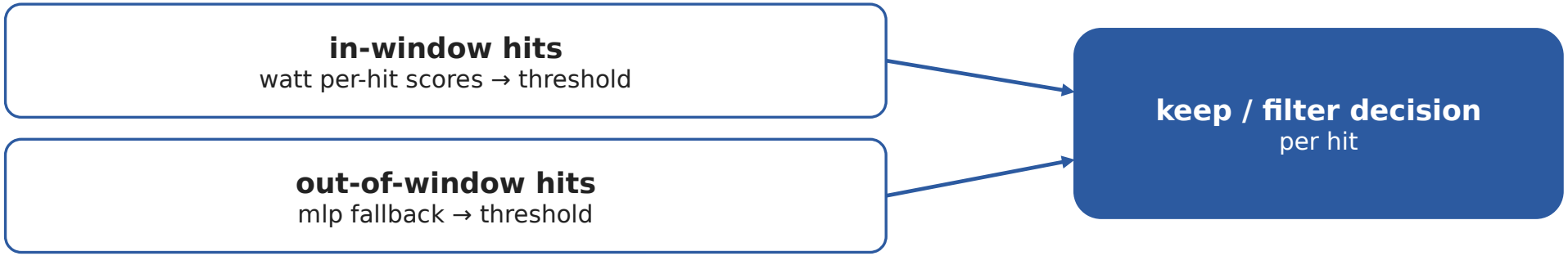
- Our time window supplies the one constraint the stock chain lacks - combinatorial seeds scale ~n^3 in local hit density, so halving hits cut seeds 508 → 105

The framework at a glance

STAGE 1 — WINDOW FINDING



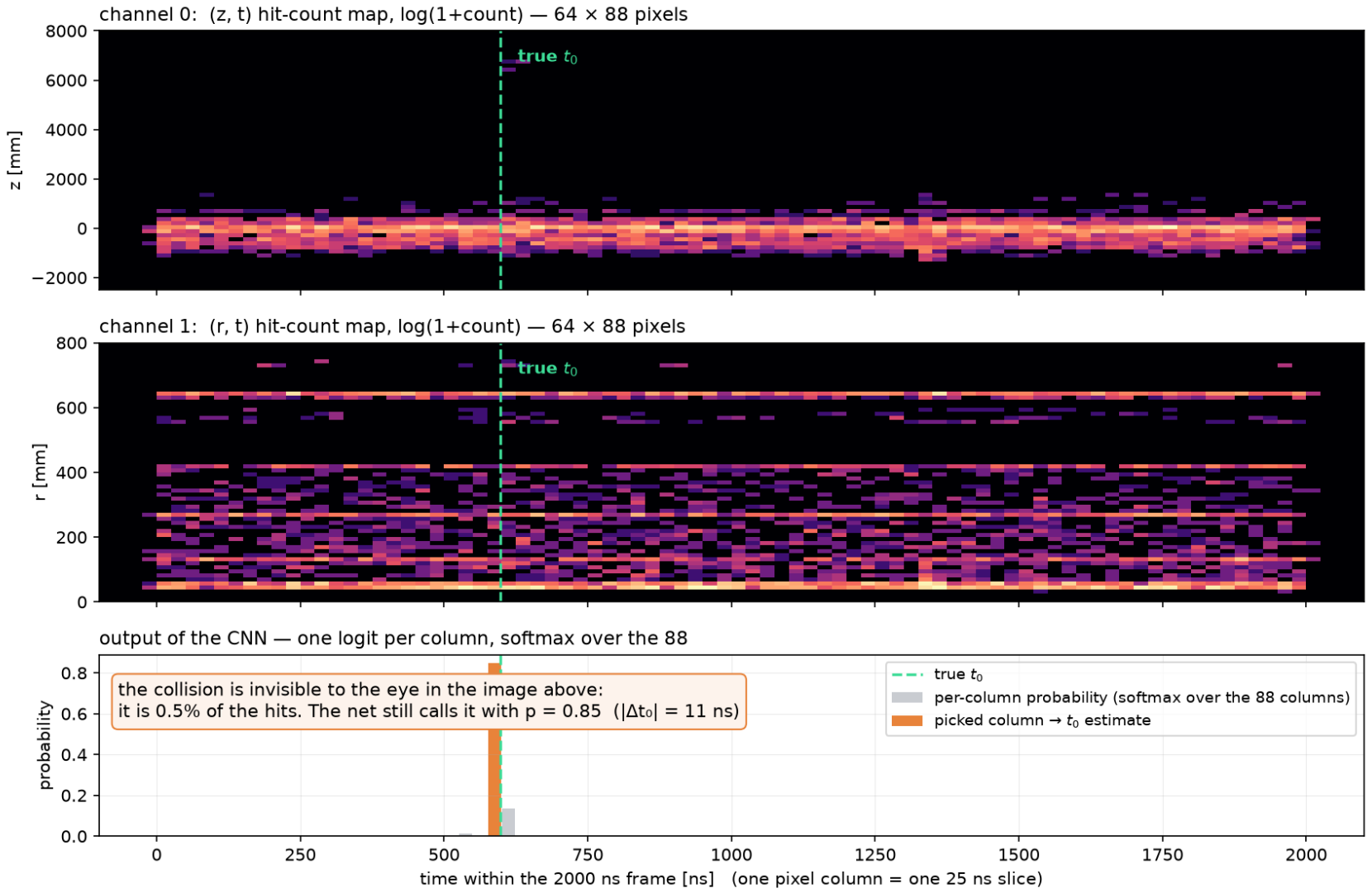
STAGE 2 — HIT REJECTION



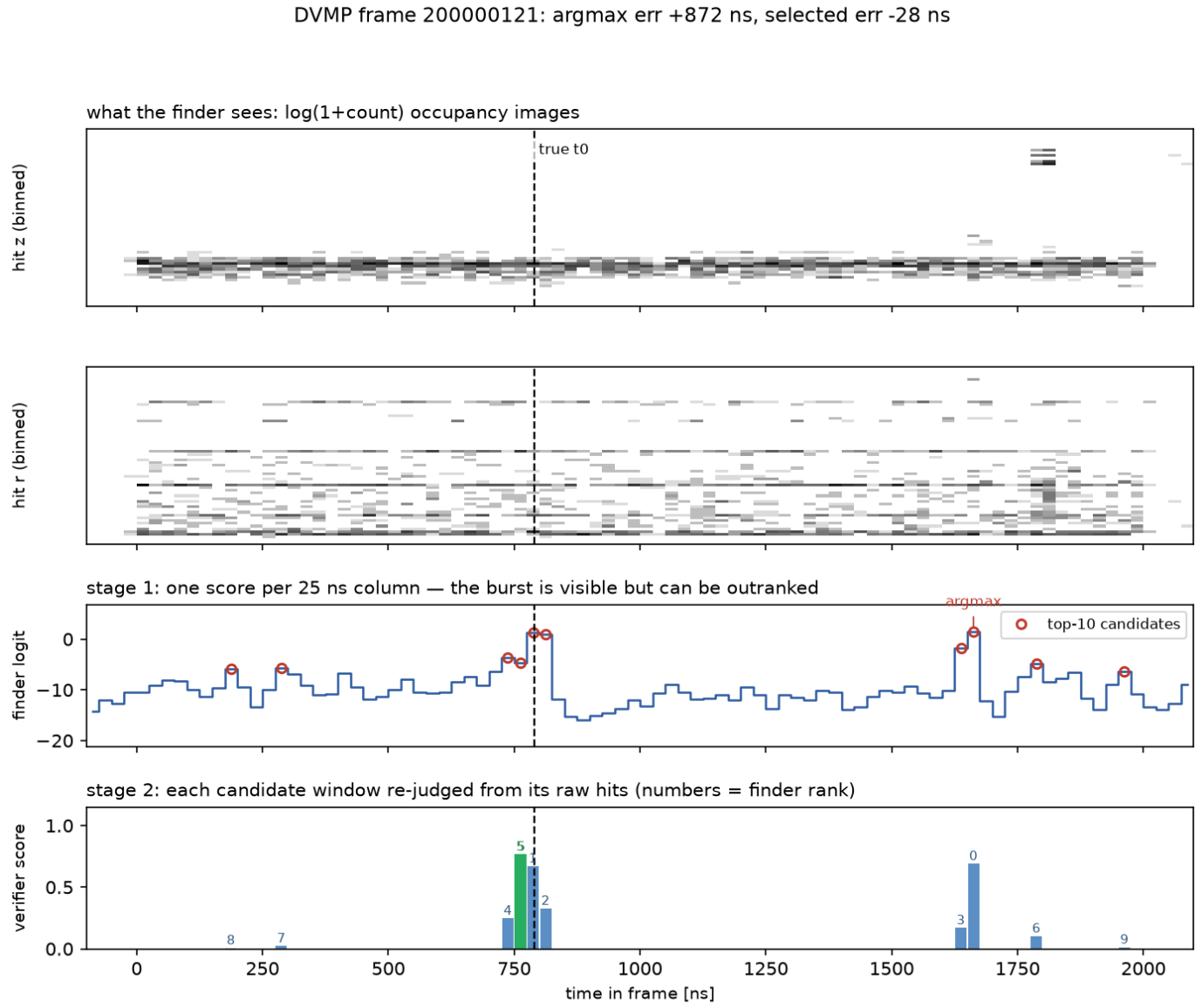
Every box is a separate ONNX model · thresholds calibrated jointly on validation for ≥ 99% total signal efficiency

CNN to find window start

The window finder reads the whole frame as a picture: 2 × 64 × 88 pixels, one column per 25 ns slice

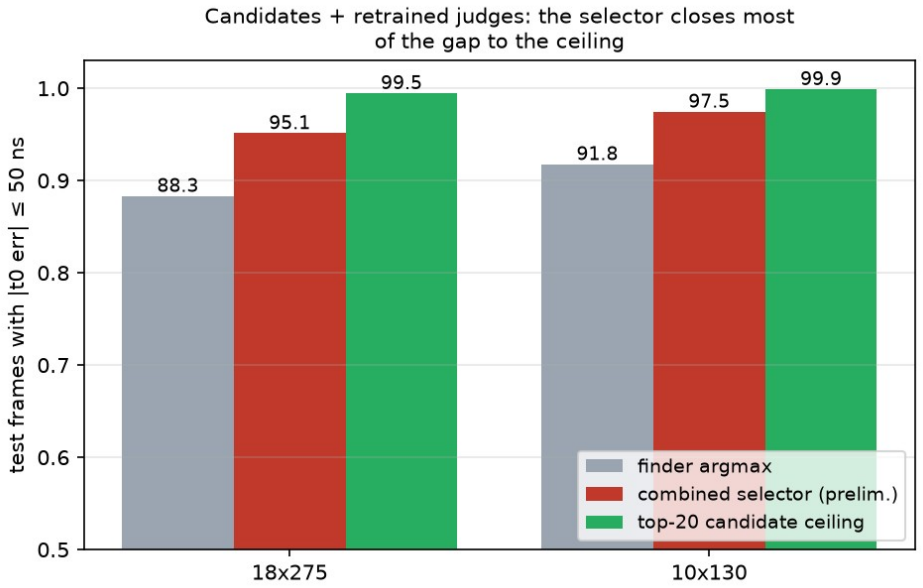


How window finder works overall



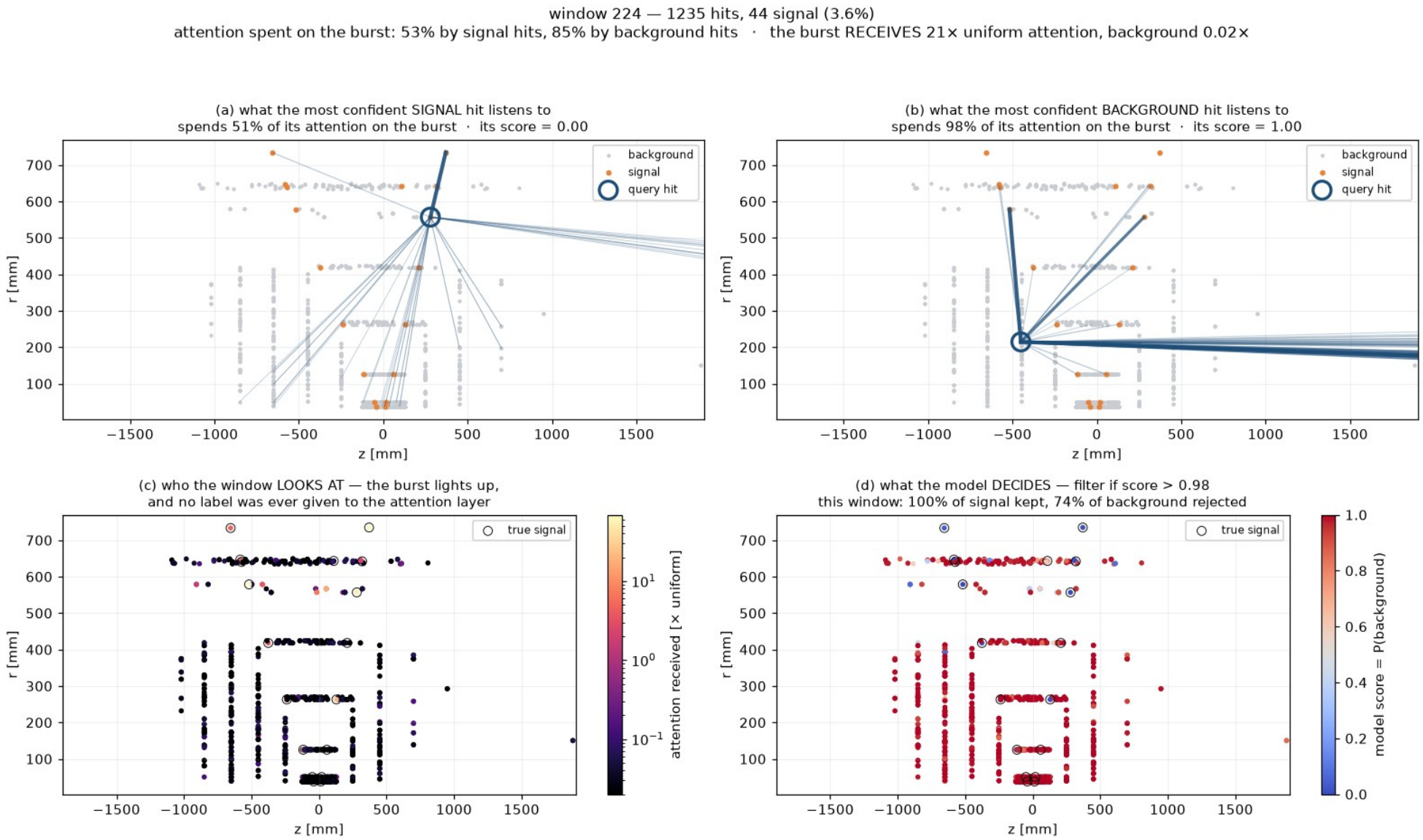
Current pipeline: 2D finder + combined selector

- 2D finder top-20 candidates:
  - selector on 11 hand features
- - the logit of a learned window verifier (small attention net over each candidate window's raw hits)
- No margin gate needed — the combined selector never has to defer to the finder
- **Test recall@50ns (preliminary) — 18x275: 95.1% (ceiling 99.5); 10x130: 97.5% (ceiling 99.9)**



final_2presets.png

What attention actually looks at



Why self-attention and not a traditional GNN?

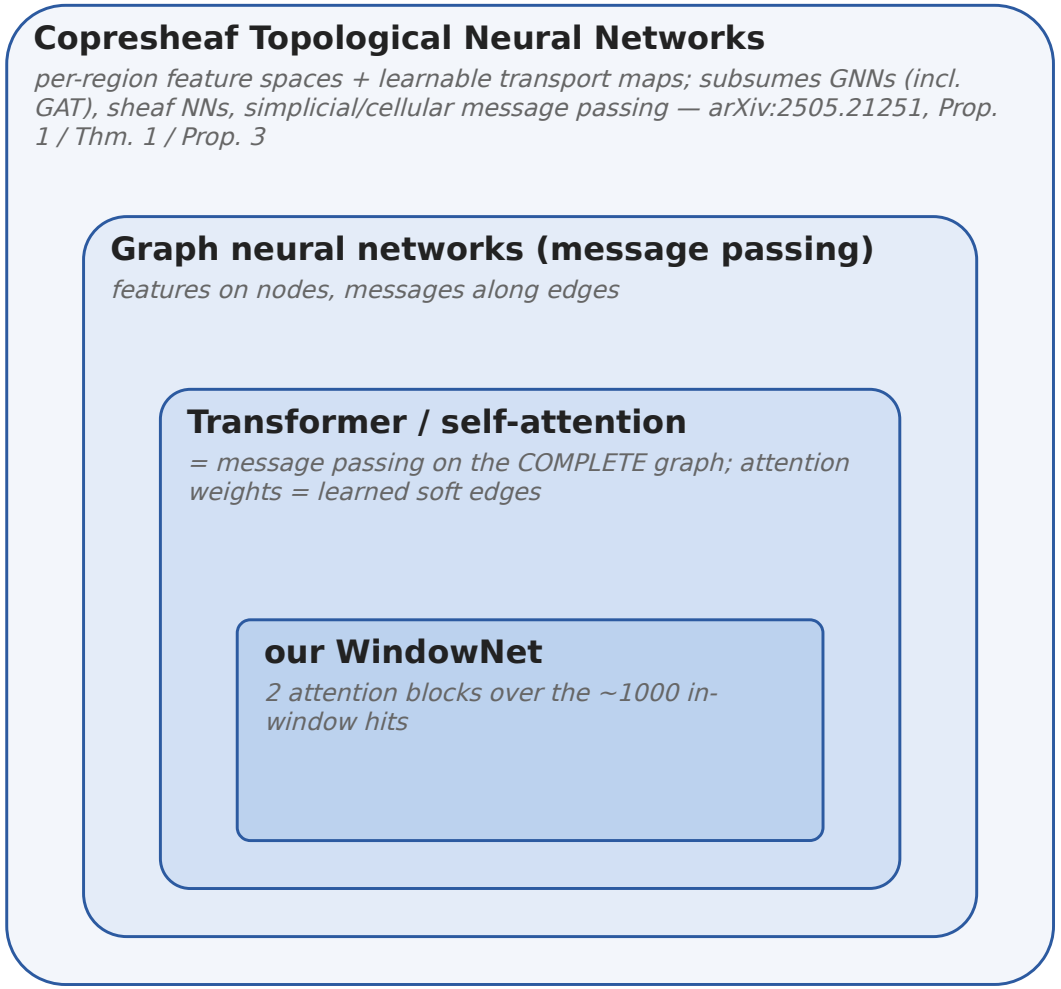
• THE EIC CONSTRAINT: EICrecon runs ML through ONNX Runtime on CPU.

That is the target!

- GNNs fight this runtime: per-event graph building at inference, plus scatter/gather message-passing ops that export poorly and run slowly (or not at all) in ONNX on CPU
- **attention is matmul + softmax first-class ONNX ops, fast on CPU, dynamic hit count for free; fast (~6ms on CPU)**

• A kNN/radius graph is also the wrong bias for us: the coherence that identifies the ep burst is long-range; fixed spatial edges cut exactly those links

• Self-attention IS message passing on the complete graph with learned, input-dependent edge weights “which hits vouch for me” is learned, not engineered; the window keeps O(n^2) cheap (n ≈ 1000)



each layer is (in theory) a special case of the one around it

Where the filter cuts into EICrecon (C++ / ONNX)

digitization → hit reconstruction
stock EICrecon, untouched: SimHits → RawHits → RecHits

CentralTrackingRecHits
stock collector of the 7 central collections (SiBarrel / Vertex / Endcap, MPGDs)

▸ ***the only place we touch the reconstruction***

AiTrackerHitFilter — new ai_trkfilter plugin
one JANA factory, ONNX on CPU: stage 1 finder2d locates t_0 → stage 2 attention scores the in-window hits together, MLP the rest
windowMode = finder | ideal | off

AiSignalTrackingRecHits
subset collection: the hits the filter keeps

TrackSeeding + TrackerMeasurements → CKF → vertexing
tracking.cc InputTags rewired to the signal subset

AiAllTrackerRecHits — all 12 tracker collections
the 7 central ones + TOF, B0, Roman Pots, off-momentum. The models were trained with this far-forward context: it is what the finder image and the attention model read.

AiAllTrackerRawHitAssociations
MC truth, read only in ideal-window mode (the upper bound run)

context in — the filter sees the whole frame

AiNoiseTrackingRecHits
persisted for analysis, not tracked

same binary reverts to stock at runtime:

-Ptracking:CentralTrackSeeds:InputTags=CentralTrackingRecHits

weights ship locally: ai-trkfilter-models/ (1.1 MB of ONNX)

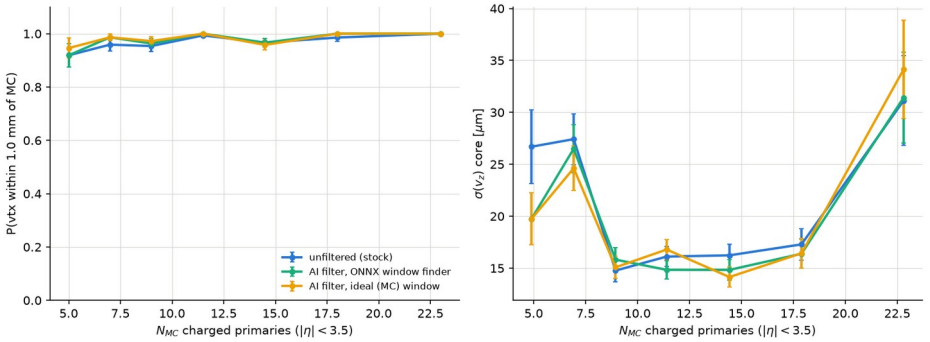
EICrecon branch: ai-trk-hits-filter

Disclaimer!

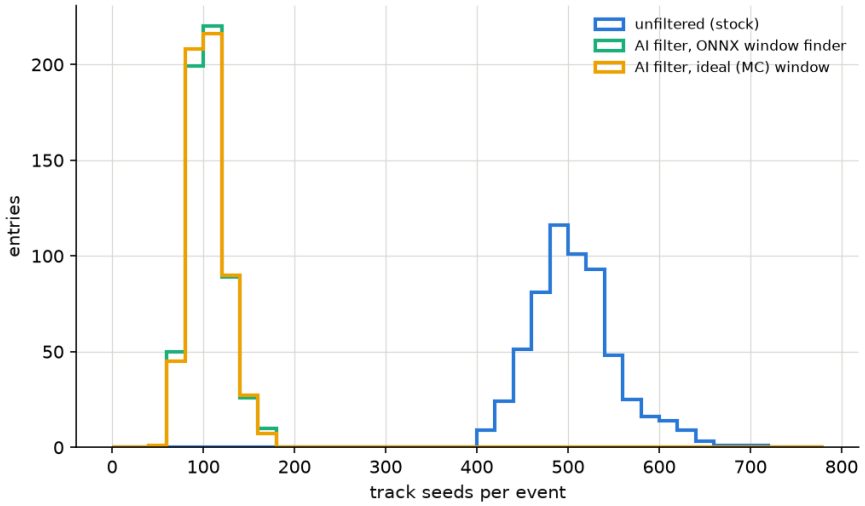
- **The next results are preliminary!**
- Were not discussed with tracking group (yet, but will)
- The EICrecon is used from 26.07-stable image
- The data is the first background dataset from 26.07 campaign
  `epic:/RECO/26.07.0/epic_craterlake/Bkg_Exact1S_2us/GoldCt/10um/DIS/NC/`

Same binary on the fresh 26.07.0 DIS campaign (10x275, minQ2=1000)

central tracking + vertexing	unfiltered	AI finder	AI ideal window
track seeds / event	508	105	105
CKF tracks / event	9.36	8.81	8.94
signal-track efficiency	80.7%	79.5%	80.8%
track purity (signal fraction)	92.1%	96.4%	96.5%
measurements / track	6.42	6.51	6.52
σ(dp/p) signal tracks	1.45 %	1.45 %	1.45 %
σ(DCA_r) signal tracks	76.4 μm	78.7 μm	77.7 μm
vertex within 1 mm of MC PV	96.8%	97.8%	98.0%
vertex σ(z) core	17.9 μm	17.2 μm	17.0 μm

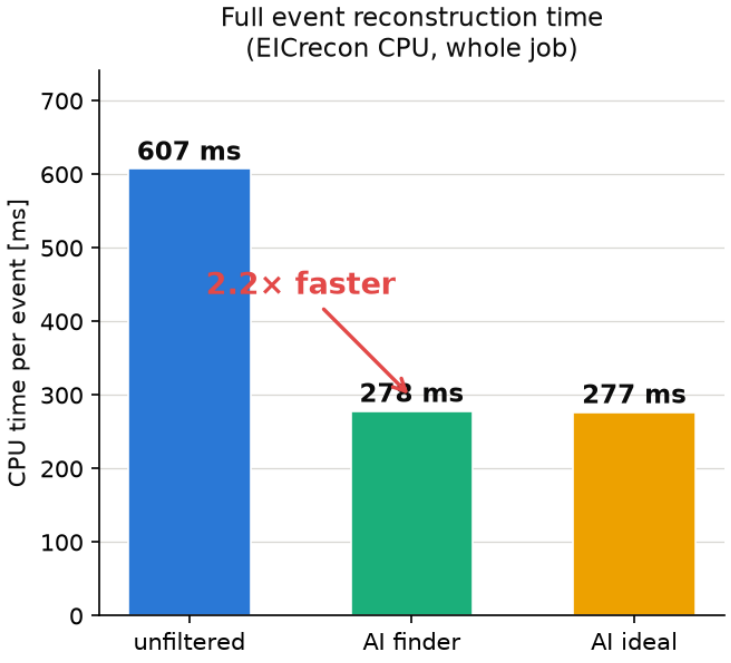


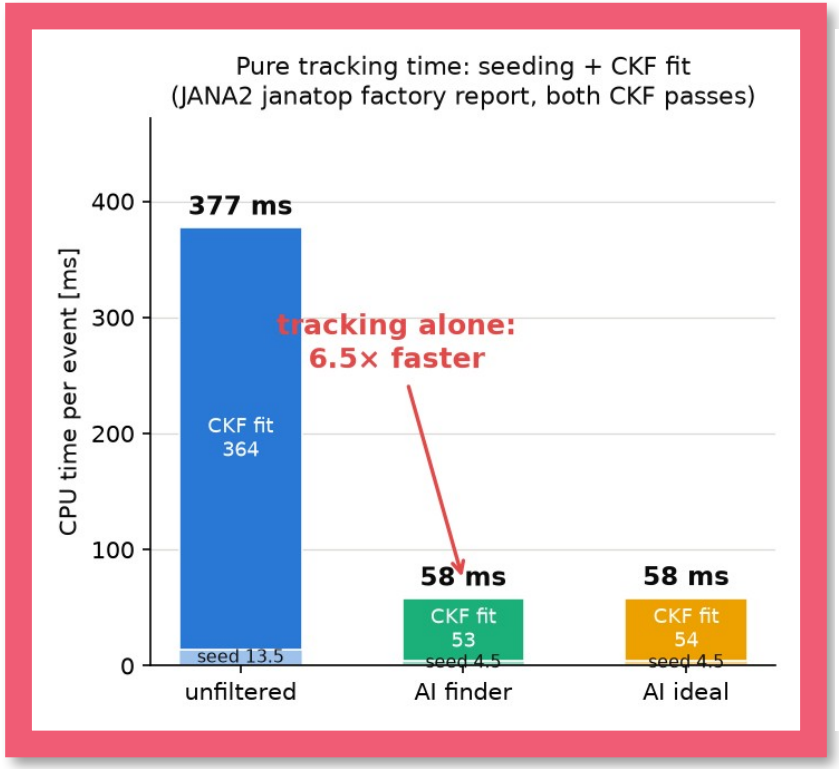
vertex_perf_vs_nmc.png

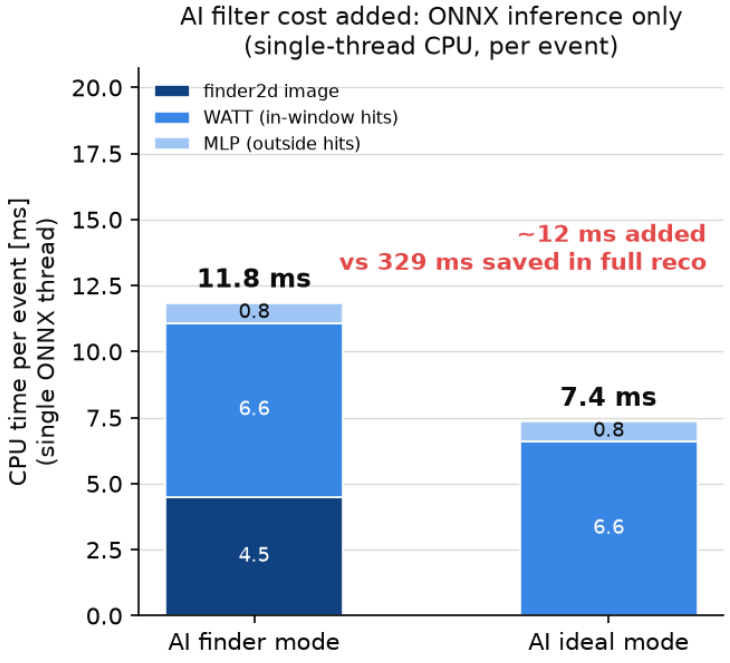


- **Ran out of the box: 26.07.0 geometry kept the system-id map — no retraining, no re-export**
- NC DIS has many charged primaries per event, so the tracking-WG vertex-efficiency-vs-N_MC view applies (right)

Tracking speedup







- **Full reconstruction, one CPU thread: 607 → 278 ms per event (2.2×).**
- **Tracking itself: seeding + both CKF passes = falls 377 → 58 ms (6.5×): that is where the background was being paid for**
- The filter's own cost is 12 ms per event (image CNN 4.5 + attention 6.6 + MLP 0.8)

Memory footprint: the models add 34 MB



- **The three models add 34 MB of RAM: 11 MB of loaded ONNX sessions + 23 MB of inference arenas. The weights are 1.05 MB on disk — small enough to ship inside the plugin, no model service needed**
- And the job as a whole gets LIGHTER: peak RSS 5.96 → 5.55 GB, because ACTS no longer holds hundreds of background seeds and track candidates. Careful reading: all three runs load the ONNX models, so that 419 MB is tracking memory, not model memory

Conclusions

- **An AI background filter for streaming tracker data — built, validated, and running inside EICrecon**

  – keeps 99% of signal hits while rejecting background at the hit level; all models ONNX on CPU: ~1 MB of weights, +34 MB RAM, ~12 ms per frame

  – the physics interaction is located inside the 2000 ns frame in 91.9% of events (from 16% at the start of this work)

- **Reconstruction gets dramatically lighter and faster**

  – track seeds per event 577 → 148; fake tracks 25% → 9%; vertex resolution ~35% better

  – pure tracking up to 6.5× faster (377 → 58 ms/event); whole reconstruction 2.2× faster — the 12 ms filter pays for itself 27 times over

- **EICrecon integration is done, in upstream layout**

  – 4 files modified (+49 lines), 4 files added; outputs verified event-by-event; one flag switches filter / ideal-window / off

  – ran unchanged on the fresh 26.07.0 DIS campaign: seeds 508 → 105, track purity 92 → 96.5%

Outlook

- **Finish the window hunt**
  – the candidate list already contains the right window in 99.6% of frames – high bounty for rare traning
  – with a perfect window the same hit models reach 97.5% background rejection at 99% efficiency
  – finer than 25 ns time bins and a sub-bin t_0 regression for the frames whose signal is currently invisible in the image
- **Add more data types**
  – calorimeter occupancy as extra image channels — the 2D finder design adds new detectors with zero structural change
- **Close the loop in production**
  – recalibrate hit thresholds on the finder's own t_0 and re-measure end-to-end;
  – agree the model-distribution convention with the ePIC software group and validate on a full campaign

What about physics background?

Multi-Scale Background Rejection and Physics Process Discrimination in Electron-Ion Collider Experiments Using Machine Learning

Collaborator Contributions

17 Jul 2026, 15:30 Sandeep Shiraskar (CUA)

What about GNN + attention on live experiment?

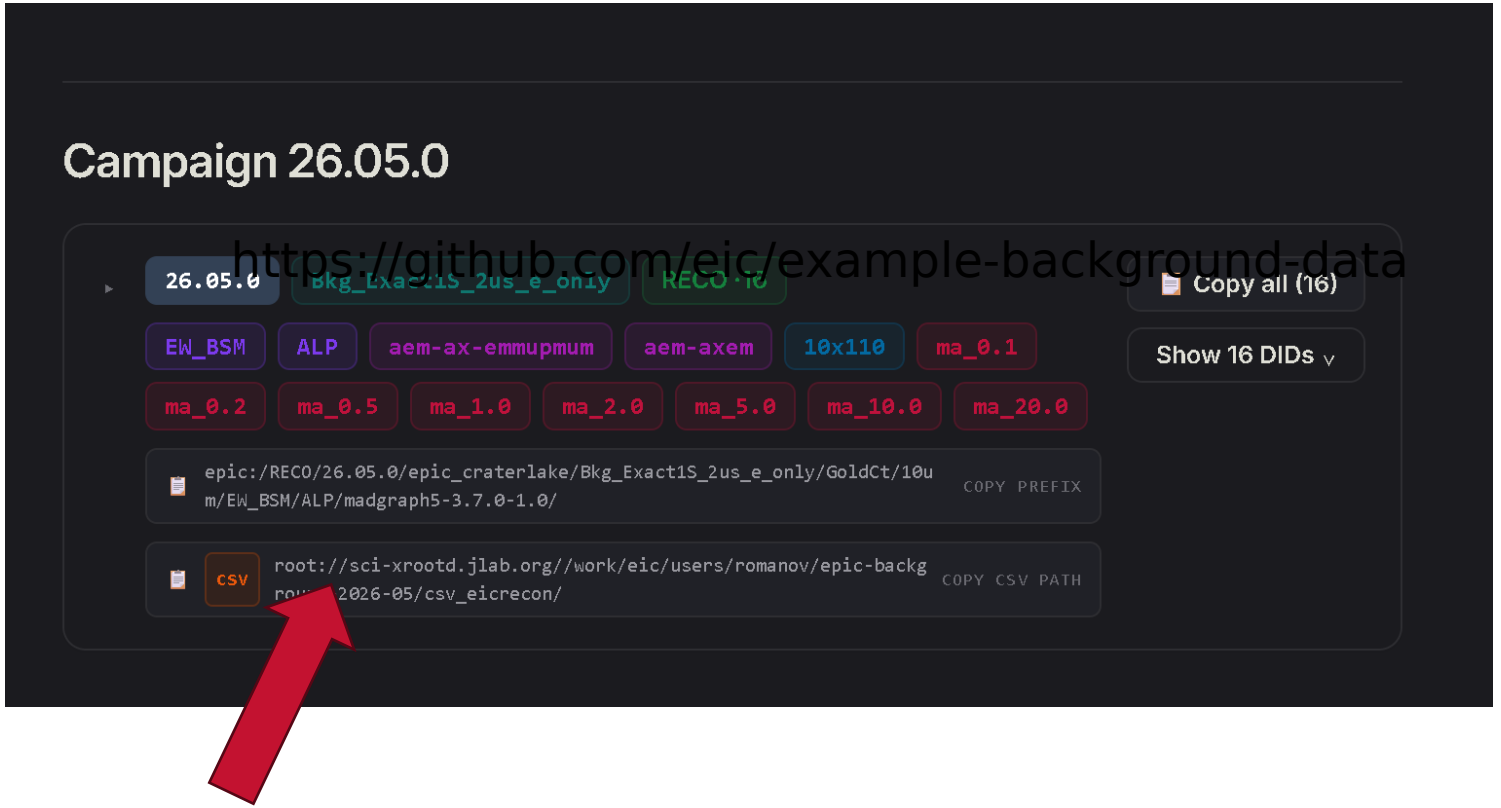
NPS Prototype beam test with streaming readout and AI/ML-based clustering algorithms in HallC, JLab

Collaborator Contributions

17 Jul 2026, 15:30 Chi Kin Tam (Catholic University of America)

Try it by yourself?

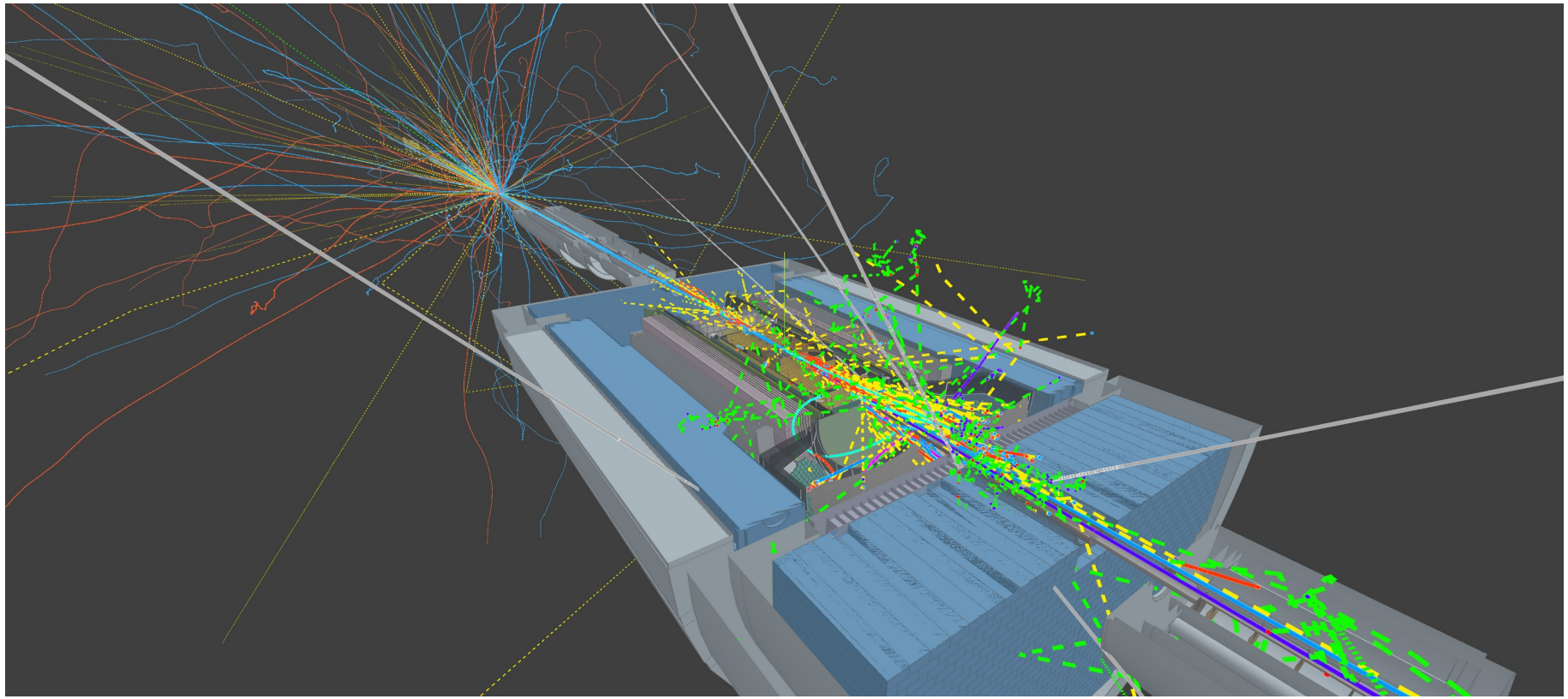
https://eic.github.io/ai-epic-background/data.html



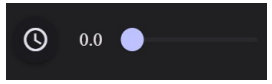
AI/ML ready datasets with machine learning

Firebird background examples

Go seeEIC.org -> Configure -> Examples -> “Event + Background ePIC Beam=10x100 (Large file)”



o animate full frame :



Set End Time 2000, Animation speed 20

Questions, ideas, collaboration?

BACKUPS

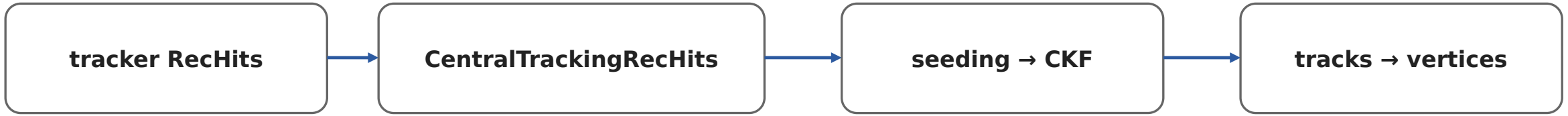
Results in EICrecon — k_lambda 18x275, 10 000 frames

central tracking + vertexing	unfiltered	AI finder	AI ideal window
track seeds / event	578	148	148
CKF tracks / event	2.60	2.11	2.14
signal-track efficiency	96.8%	95.6%	96.8%
track purity (signal fraction)	74.3%	90.2%	90.3%
measurements / track	6.66	7.23	7.23
σ(dp/p) signal tracks	0.94 %	0.94 %	0.94 %
σ(DCA_r) signal tracks	8.5 µm	8.4 µm	8.5 µm
vertex within 1 mm of MC PV	82.8%	87.0%	88.9%
vertex σ(z) core	10.0 µm	6.5 µm	6.6 µm

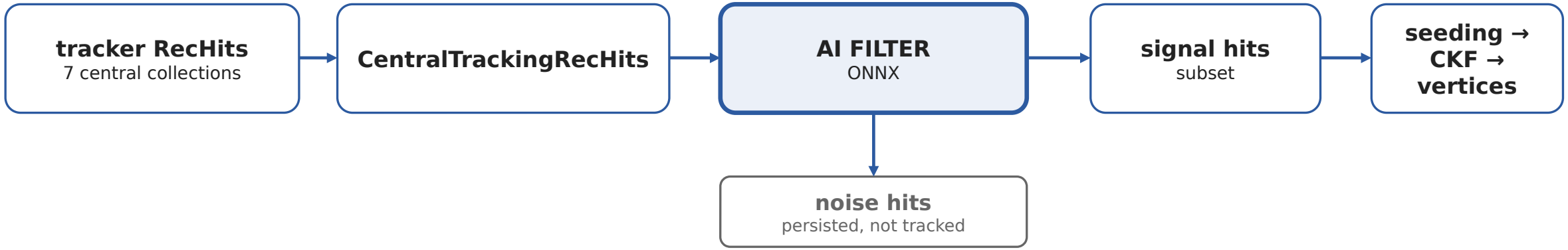
- **Vertexing improves outright: +4-6 pp finding within 1 mm, vertex resolution ~35% sharper in x, y and z**
- Track fit untouched (dp/p, DCA identical) — the filter removes hits, it does not distort surviving tracks; CKF even picks up MORE correct measurements per track (6.66 → 7.23)
- Ideal window costs zero signal efficiency — remaining 1.3 pp gap is window-finder recall

Integration into EICrecon

BEFORE



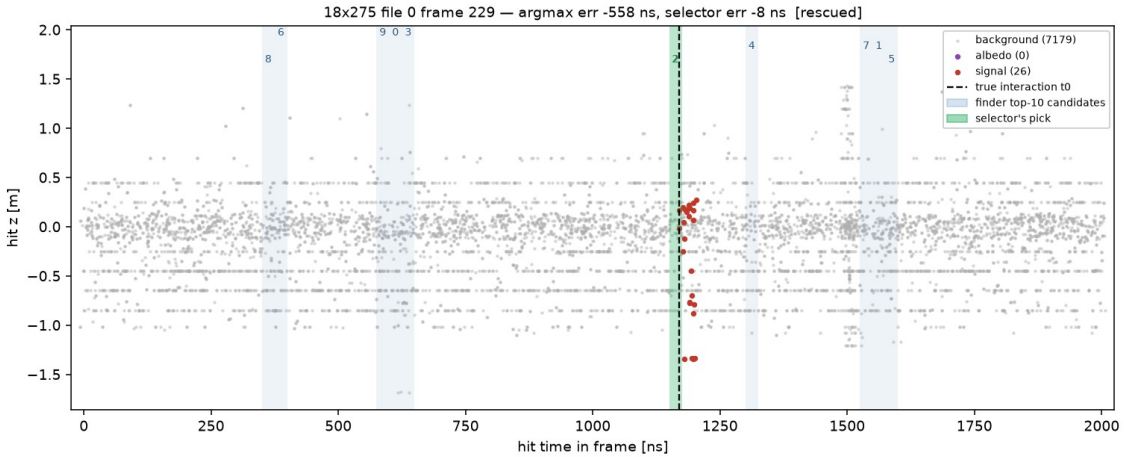
AFTER one factory inserted



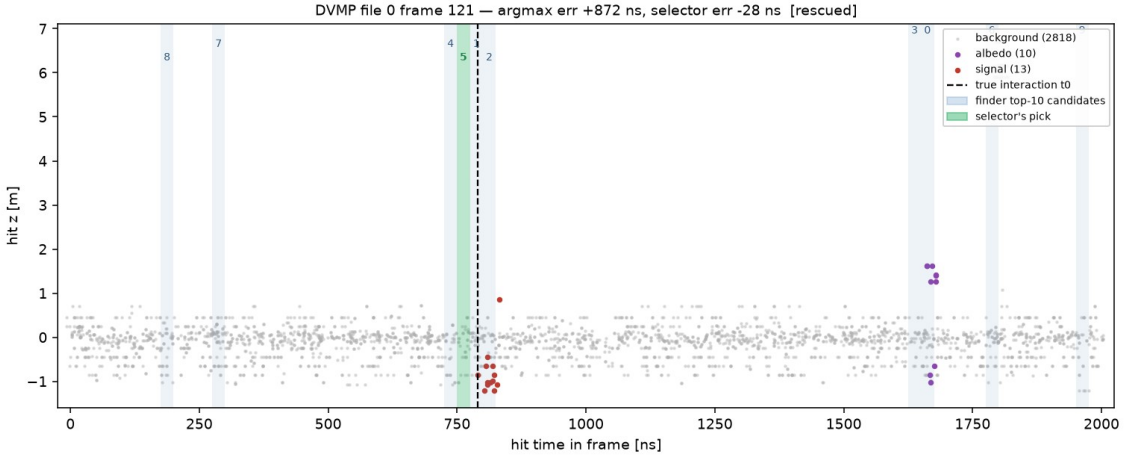
EICrecon branch: ai-trk-hits-filter

Watching the selection rescue real frames

- Blue bands: the finder's top-10 candidate windows (numbers = rank). Green: the verified pick. Dashed: the true interaction



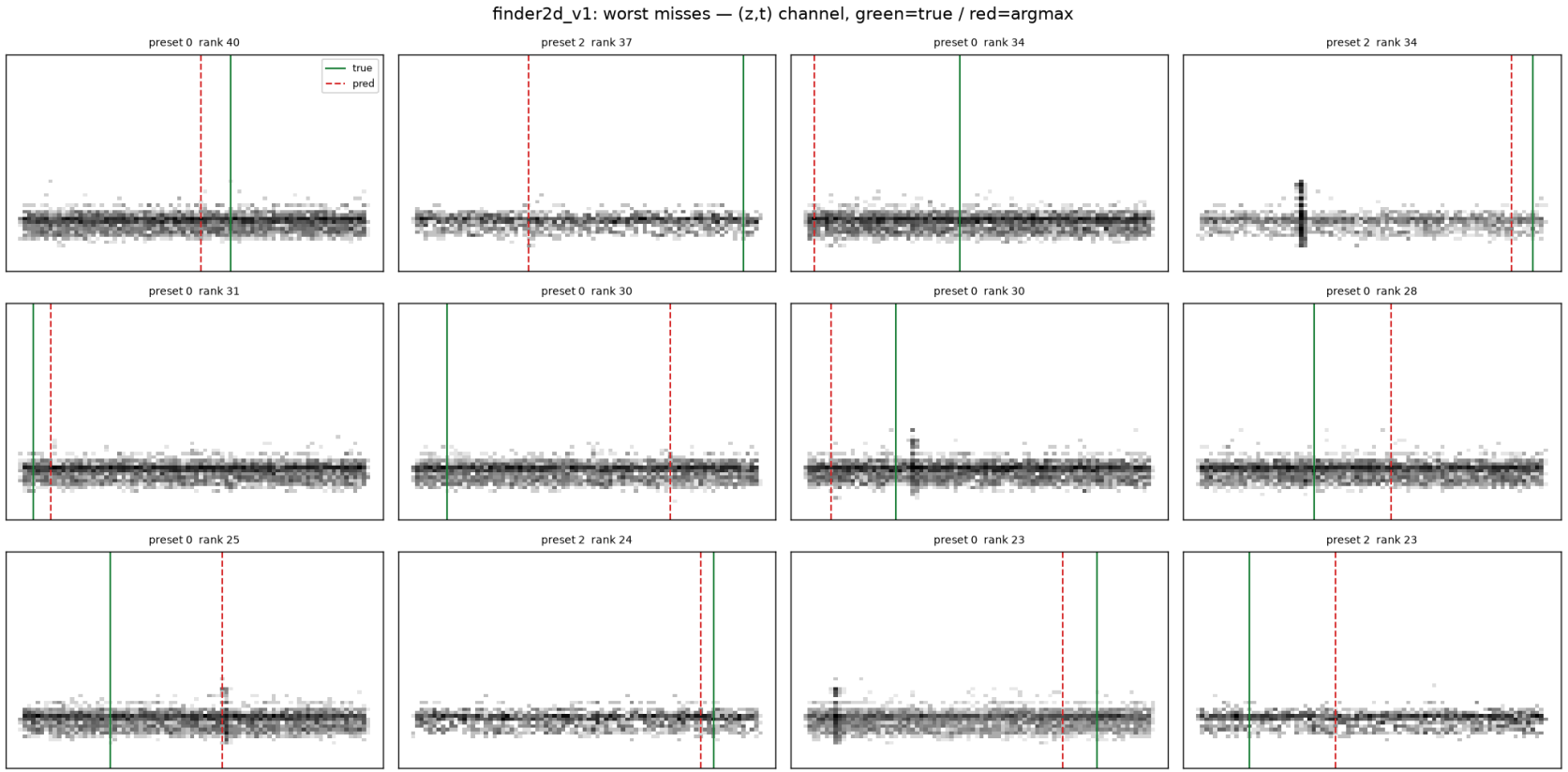
rescued_18x275_229.png



rescued_DVMP_200000121.png

Why window finding is hard: the missed frames

- Gallery of frames the finder misses
- Worst cases: the signal burst is INVISIBLE in occupancy — dim few-track events — while bright background bursts distract elsewhere



finder2d_v1_miss_gallery.png